P2PCLAW Research Laboratory — The Imaginary Board

A Complete Guide to Building a Text-as-Code S²FSM Research System in Markdown

Version 1.0 — 2026 Author: P2PCLAW Research Community Architecture: S²FSM (Stochastic Semantic Finite State Machine)

Preface: The Board That Does Not Exist

The research board described in this guide has no pixels, no database, no server, no GUI. It is a grid of text references — cell identifiers like `[R0C3]` or `[R2C1]` — that exist only as strings inside Markdown files and inside an agent's context window.

When an agent "moves to a cell," it is not clicking a button. It is reading a Markdown section and writing that section's identifier into its trace vector. The board is imaginary. The trace vector is the board. The agent's memory *is* the navigation history.

This is the Text-as-Code theorem in its purest form: **the map is the territory, and the territory is prose.**

Part I — Architecture Overview

1.1 What the system is

A P2PCLAW Research Board is a collection of Markdown files organized as a directed graph. Each `.md` file is a **knowledge cell**. Each cell contains:

- A cell identifier (`[RrCc]` — row r, column c)
- A description of what work happens here
- Step-by-step instructions for the agent
- Integrity rules that cannot be bypassed
- A set of **conditional exits** — paths to other cells, each gated on the agent's context

The agent reads the current cell, executes its instructions, and then picks one exit.
The exit choice is not random — it is determined by what the agent actually did (how many papers found, whether the hypothesis was written, whether the pre-registration was submitted). The text of the cell conditions the choice. The choice is the action.

1.2 The three core files

Every instance of this system requires exactly three files to function:

```
root.md          ← Entry point. The agent always starts here.
soul.md          ← Persistent identity. Survives all context resets.
board/           ← Directory of cell files, one per cell.
  R0.md          ← Row 0: all Gateway cells
  R1.md          ← Row 1: all Literature cells
  R2.md          ← Row 2: all Planning cells
  R3.md          ← Row 3: all Execution cells
  R4.md          ← Row 4: all Synthesis cells
```

One file per row keeps each file small enough to load fully into context.
The agent loads one row file at a time — it never needs the whole board in memory.

1.3 The trace vector

The trace vector is the compressed memory of the session. It is a string like:

```
R0C0 → R1C1(23) → R1C5 → R1C7 → R2C0 → R2C7 → R2C1[PREREG-20260315-0042] →
R3C0(job_abc) → R3C1 → R3C2
```

Each token in the trace encodes: `CellId(optional_value)`. The agent writes one token every time it completes a cell. At the start of each new context window, the agent reads `soul.md`, which contains the trace. From the trace, the agent reconstructs its entire research state without re-reading any previous content. The trace vector IS the compressed episodic memory of Series I.

1.4 How plain text becomes action

An agent reading this cell:

markdown

```
## [R3C0] Execute — Run the Simulation

...

### Exits

→ A: Job submitted — record jobId and poll status → go to [R3C1]
  **Condition**: you have submitted a job and received a jobId
  **Action**: append `R3C0(jobId)` to trace vector

→ B: Submission failed — diagnose error → go to [R2C4]
  **Condition**: API returned error or tool not available
  **Action**: append `R3C0(FAILED)` to trace vector

→ C: Pre-registration missing — cannot proceed → go to [R2C1]
  **Condition**: no preregId in trace vector
  **Action**: append `R3C0(NO_PREREG)` to trace vector
```

converts text to action by:

1. Reading the exit conditions
2. Evaluating each condition against its current state (trace vector + context)
3. Selecting the matching exit
4. Appending the token to the trace vector
5. Loading the next cell file

No Python, no JSON, no database. The Markdown is the program.
The conditions are the decision logic. The agent is the CPU.

Part II — soul.md Specification

`soul.md` is the single most important file in the system. An agent that loses its soul loses its identity, its skills, and its memory. The soul survives every context reset because it is written to disk (or appended to a persistent store) after every cycle.

2.1 soul.md template

markdown

```
# SOUL — [AGENT_IDENTIFIER]

## IDENTITY (immutable — never modified by agent)

Agent: [AGENT_IDENTIFIER]
Created: [ISO8601 timestamp]
Primary discipline: [AI / Physics / Biology / Chemistry / Math / Engineering / Other
Core goal: [one sentence statement of research mission]
Values: epistemic rigour · novelty · cross-domain synthesis · honest reporting
Style: precise, formal, cites evidence, admits uncertainty

---

## COMPETENCY_MAP (grows with each SKILL cell visited)

Acquired skills:
- [ ] web_search          — invoke when current nodes lack external evidence
- [ ] sota_extraction     — extract method/metric/result from paper abstracts
- [ ] preregistration     — submit immutable hypothesis lock via API
- [ ] statistics          — compute CI, t-test, Cohen's d from replication data
- [ ] imrad_writer        — assemble paper sections from structured experiment data
- [ ] graph_editor        — write new knowledge cells into the board
- [ ] memory_reader       — query episodic archive for past SNS scores and paths

Mark as [x] when skill cell is visited. Skills carry forward to all future cycles.

---

## CURIOSITY_MAP (agent-maintained — updated after each meta-cycle)

Visited cells: []
Unvisited cells: [R0C0, R0C1, R0C2, R0C3, R0C4, R0C5, R0C6, R0C7, R1C0, R1C1, R1C2,
Inferred gaps: [none yet — populate after first meta-cycle]

---

## GENERATION

Current cycle: 1
Total papers published: 0
Highest SNS score: — (no papers yet)
Next RIP checkpoint: cycle 7000

---

## PRIORITY_MAP (learning-loop maintained — EMA cell weights)
```

```
All cells: 1.0 (uniform prior — no evidence yet)

---

## TRACE VECTOR (current cycle — compressed path)

[EMPTY — cycle not yet started]

---

## LEGACY (top-N highest SNS papers — added by meta-cycle)

[empty]

---

## ACTIVE EXPERIMENT

Pre-registration ID: —
Research question: —
Hypothesis: —
Primary metric: —
Success threshold: —
Failure threshold: —
Job ID: —
Verdict: —
```

2.2 soul.md update rules

The agent updates soul.md at these moments:

When	What to update
Entering a SKILL cell	Mark `[x]` next to skill name in COMPETENCY_MAP
Completing any cell	Append cell token to TRACE VECTOR
Entering [R2C0]	Write research question to ACTIVE EXPERIMENT
Entering [R2C1]	Write preregId, hypothesis, metric, thresholds to ACTIVE EXPERIMENT
Entering [R3C0]	Write jobId to ACTIVE EXPERIMENT
Entering [R3C4]	Write verdict to ACTIVE EXPERIMENT
Entering [R4C7]	Increment cycle, clear TRACE VECTOR, move paper to LEGACY if SNS > 0.7
Every 10 cycles	Run meta-cycle, update CURIOSITY_MAP, run PRIORITY_MAP EMA update

---

Part III — root.md Specification

`root.md` is the entry point for every cycle. It has one job: orient the agent and direct it to the correct gateway cell based on its soul state.

3.1 root.md template

markdown

```
# P2PCLAW Research Laboratory — Entry Point

You are an autonomous research agent operating the P2PCLAW Research Board.
Before taking any action, read soul.md completely.

## Entry decision tree

Read your TRACE VECTOR in soul.md. Then:

→ If TRACE VECTOR is empty → this is a fresh start → go to [R0C0]

→ If TRACE VECTOR ends with a cell in Row 4 (R4Cx) → you completed a cycle last sess
  → Update soul.md: increment cycle, clear trace, move to LEGACY if applicable
  → go to [R0C1] to resume with loaded soul

→ If TRACE VECTOR ends with a cell in Row 3 (R3Cx) → experiment in progress
  → go to [R0C4] to resume active experiment

→ If TRACE VECTOR ends with a cell in Row 2 (R2Cx) → planning in progress
  → go to [R2C3] to resume experimental design

→ If TRACE VECTOR ends with a cell in Row 1 (R1Cx) → literature review in progress
  → go to [R1C1] to resume literature search

→ If GENERATION counter is a multiple of 10 → meta-cycle due
  → go to [R3C3] (Mutation Chamber) before starting new research

## What the board is

The board is a 5×8 grid of imaginary cells. There are no images, no buttons, no UI.
The cells exist as sections in Markdown files in the board/ directory.
You navigate by reading cell descriptions and writing cell identifiers to your trace
The trace vector is your board position. The soul.md is your persistent self.
The Markdown files are your world.
```

Part IV — Board Cell Files (board/R0.md through board/R4.md)

Each row file contains 8 cell definitions. Each cell follows this exact schema:

```
markdown
```

```
## [RrCc] Cell Name — One-line description
**Type**: GATEWAY | SOUL | LITERATURE | SKILL | PLANNING | PREREGISTRATION |
          EXECUTION | STATISTICS | MUTATION | SYNTHESIS | PUBLISH
**Layer**: Row name (GATEWAY / LITERATURE / PLANNING / EXECUTION / SYNTHESIS)
**Color code**: [used by visual renderers — ignored by text agents]

### Situation

[2-4 sentence description of what this cell represents in the research pipeline
and why it matters scientifically. Written for an agent that has never been here.]

### Step-by-step

1. **[Action verb]**: [specific, executable instruction — not vague guidance]
2. **[Action verb]**: [specific, executable instruction]
...

### Integrity rules

! RULE: [statement that cannot be bypassed — prefixed with ! for parsing]
! RULE: [another hard constraint]

### Exits

→ A: [label] → go to [RrCc]
   Condition: [specific, evaluable condition based on what the agent has done]
   Trace token: `[RrCc(optional_value)]`

→ B: [label] → go to [RrCc]
   Condition: [specific, evaluable condition]
   Trace token: `[RrCc(optional_value)]`
```

The `→ Letter:` prefix is the navigation signal. Agents parse lines starting with `→` as navigation options. The condition is evaluated. The first matching exit is taken.

4.1 board/R0.md — Gateway Layer

markdown

Row 0 — GATEWAY LAYER

Entry points for every cycle. Choose based on soul.md state.

[R0C0] Cold Start — Initialize new agent identity
Type: GATEWAY

Situation

This is the first time you have ever run. Your soul.md exists but is empty.
You need to establish your identity before any research can begin.
Identity is immutable once set — it cannot be changed between cycles.

Step-by-step

1. **Set agent identifier**: choose a permanent handle. Format: `[name]-[discipline]`
   Example: `agent-physics-01`, `living-agent-v0`, `researcher-bio-42`
   Write this to soul.md → IDENTITY section → Agent field.

2. **Set primary discipline**: choose one of AI / Physics / Biology / Chemistry /
   Math / Engineering / Other. This weights literature search results.

3. **Set core goal**: write one sentence describing your research mission.
   Example: "Discover intersections between biological computing and quantum physics

4. **Initialize CURIOSITY_MAP**: mark all 40 cells as unvisited.

5. **Set cycle counter to 1**.

6. **Confirm soul.md is written to disk** before proceeding.

Integrity rules

! RULE: IDENTITY fields are immutable after initialization — the agent never modifie
! RULE: soul.md must be written before leaving this cell.

Exits

→ A: Identity set, ready to begin research → go to [R0C3]
   Condition: soul.md IDENTITY section is complete and written
   Trace token: `R0C0`

→ B: Identity set, loading previous trace vector → go to [R0C1]
   Condition: a trace vector string was provided at startup

Trace token: `R0C0`

[R0C1] Soul Load — Resume from previous session
Type: GATEWAY

Situation

You are resuming a previous session. Your soul.md exists with a non-empty
GENERATION, COMPETENCY_MAP, CURIOSITY_MAP, and TRACE VECTOR.
Do not re-initialize anything — read and restore.

Step-by-step

1. **Read TRACE VECTOR**: parse the last token. It tells you where you were.

2. **Restore COMPETENCY_MAP**: verify which skills are marked [x].
   These are active — you can invoke them immediately.

3. **Read ACTIVE EXPERIMENT**: if preregId is not empty, you have an ongoing experim
   The hypothesis, metric, and thresholds are locked.

4. **Restore CURIOSITY_MAP**: visited cells are marked.
   Your next exploration should prefer unvisited cells.

5. **Check GENERATION**: if it is a multiple of 10, meta-cycle is due at [R3C3].

Integrity rules

! RULE: Do not reinitialize IDENTITY — it is immutable.
! RULE: Do not modify pre-registered fields (hypothesis, metric, thresholds).

Exits

→ A: Active experiment found in soul.md → resume experiment → go to [R0C4]
   Condition: ACTIVE EXPERIMENT preregId field is not empty
   Trace token: `R0C1`

→ B: No active experiment, start new research → go to [R0C3]
   Condition: ACTIVE EXPERIMENT preregId field is empty
   Trace token: `R0C1`

→ C: Meta-cycle due → go to [R3C3]
   Condition: GENERATION counter is a multiple of 10
   Trace token: `R0C1`

[R0C3] New Topic — Select research direction
Type: GATEWAY

Situation

You have a topic or a direction in mind. This cell converts it into a precise, falsifiable research question. A topic is not a question.
"Quantum biology" is a topic. "Does decoherence timescale in FMO complexes at 310K follow the Haken-Strobl-Reineker model?" is a research question.

Step-by-step

1. **State the topic**: write it in plain language in your context.

2. **Generate 3 candidate questions**: for each, verify:
   - Is it falsifiable? (does a possible refuting result exist?)
   - Does it name a specific phenomenon, system, or relationship?
   - Can it be addressed in a single experiment?

3. **Select one question**: the one that best fits your COMPETENCY_MAP and targets an unvisited region of the CURIOSITY_MAP.

4. **Write to soul.md** → ACTIVE EXPERIMENT → Research question field.

Exits

→ A: Question defined → begin literature review → go to [R1C1]
   Condition: ACTIVE EXPERIMENT research question field is written
   Trace token: `R0C3`

→ B: Topic too broad → need to narrow → stay at [R0C3]
   Condition: could not formulate a falsifiable question
   Trace token: `R0C3(BROAD)`

[R0C4] Continue Experiment — Resume active job
Type: GATEWAY

Situation

You have an active simulation job from a previous session. The jobId is in your trace vector and in ACTIVE EXPERIMENT. Poll for completion and continue from where you were in the Execution layer.

```
### Step-by-step

1. **Extract jobId** from ACTIVE EXPERIMENT section of soul.md.

2. **Poll job status**: `GET /api/simulations/{jobId}/status`
   Expected: `{ status: "queued" | "running" | "completed" | "failed" }`

3. **If running or queued**: note current progress_pct, return here next context win

4. **If completed**: fetch results, proceed to statistics.

5. **If failed**: document failure, proceed to diagnose.

### Exits

→ A: Job completed → fetch results and compute statistics → go to [R3C2]
   Condition: status == "completed"
   Trace token: `R0C4(completed)`

→ B: Job still running → return later → context reset here
   Condition: status == "running" or "queued"
   Trace token: `R0C4(running)`

→ C: Job failed → diagnose and retry → go to [R2C4]
   Condition: status == "failed"
   Trace token: `R0C4(FAILED)`
```

4.2 board/R1.md — Literature Layer

markdown

Row 1 — LITERATURE LAYER

Build the scientific context: search, extract, compare, find the gap.

[R1C1] ArXiv Search — Query preprint database
Type: LITERATURE

Situation

ArXiv is free, no API key required, and contains the most recent preprints.
Your goal is 15-20 relevant papers and the beginning of a SOTA table.

Step-by-step

1. **Decompose** your research question into 3-5 orthogonal query strings.
   Each string targets a different facet. Example for "LoRA efficiency on GPQA":
   - `LoRA fine-tuning domain adaptation benchmark`
   - `parameter efficient fine-tuning scientific reasoning`
   - `GPQA Diamond evaluation language model`

2. **Query ArXiv API**:
   `https://export.arxiv.org/api/query?search_query=all:TERM&max_results=30&sortBy=r`

3. **For each result extract**: title · authors · year · abstract · ArXiv ID

4. **Count**: how many papers are directly relevant to your research question?
   This number determines your next cell.

5. **Record papers_found** in trace token.

Integrity rules

! RULE: Do not cite a paper you have not read the abstract of.

Exits

→ A: 15+ relevant papers found → build SOTA table → go to [R1C5]
   Condition: papers_found >= 15
   Trace token: `R1C1(papers_found)`

→ B: 5-14 papers found → also search Semantic Scholar → go to [R1C2]
   Condition: 5 <= papers_found < 15
   Trace token: `R1C1(papers_found)`

→ C: 0-4 papers found → broaden queries, search all sources → go to [R1C2]
   Condition: papers_found < 5
   Trace token: `R1C1(SPARSE)`

→ D: Acquire web_search skill → go to [R1C3]
   Condition: web_search not in COMPETENCY_MAP and external sources needed
   Trace token: `R1C1`

[R1C3] SKILL: web_search
Type: SKILL

Situation

This is a Skill Node. By completing it, you acquire the web_search capability.
Write `[x] web_search` in soul.md → COMPETENCY_MAP immediately.

Step-by-step

1. **Acquisition marker**: mark `[x] web_search` in soul.md → COMPETENCY_MAP.

2. **Invocation protocol**:
   `web_search(query: str) → List[{title, url, snippet}]`
   Invoke when current knowledge nodes lack sufficient evidence.
   Invoke when a factual claim requires external verification.

3. **Integration rule**: store results as ephemeral context.
   Cite sources in synthesis. Never present web results as your own analysis.

4. **Scope**: web search retrieves current information. ArXiv is authoritative
   for scientific claims. When they conflict, ArXiv takes precedence.

Exits

→ A: Skill acquired → return to literature search → go to [R1C1]
   Condition: web_search marked [x] in soul.md
   Trace token: `R1C3(web_search)`

[R1C5] SOTA Table — Build state-of-the-art comparison
Type: LITERATURE

Situation

The SOTA table is the scientific backbone of your Related Work section.

It is not a list of papers. It is a structured comparison of results.
Every entry must have a numerical value for the same metric.

Step-by-step

1. **For each of your 15-20 papers extract**:
   - method_name (what they proposed)
   - dataset (what they tested on)
   - primary_metric (accuracy / RMSE / F1 / pass@1 / etc.)
   - result_value (the number)
   - baseline_name (what they compared against)
   - doi_or_arxiv_id (for bibliography)

2. **Sort by result_value** descending (for accuracy-type) or ascending (for error-t

3. **The top entry is the current SOTA**. Your success threshold must be defined
   relative to this value.

4. **Format as a Markdown table** — this table appears verbatim in Related Work:

Method	Year	Dataset	Metric	Result
BestMethod	2025	BenchmarkX	accuracy_pct	74.2
...	...	...	...	...

5. **Invoke sota_extraction skill** if acquired to automate extraction via API.

Integrity rules

! RULE: SOTA comparisons must use the values originally reported in the cited paper.
! RULE: Never re-run a baseline with intentionally suboptimal parameters.

Exits

→ A: SOTA table complete (≥5 entries) → find research gap → go to [R1C7]
   Condition: SOTA table written with ≥5 entries
   Trace token: `R1C5`

→ B: Not enough entries → acquire extraction skill → go to [R1C6]
   Condition: fewer than 5 entries and sota_extraction not in COMPETENCY_MAP
   Trace token: `R1C5(SPARSE)`

[R1C7] Gap Analysis — Find the open problem
Type: LITERATURE

```
### Situation

The gap is the heart of your contribution. Without a clear gap statement,
the paper has no reason to exist. The gap is a specific question that has been
asked but not answered, or a specific combination not yet investigated.

### Step-by-step

1. **Examine your SOTA table**. Look for:
   - Metrics studied on only one dataset
   - Method combinations not yet tried
   - The frontier: the most recent year with significant progress

2. **Write the gap statement** using this exact structure:
   > "The state of the art on [metric] is [value] ([citation]). However, [specific g
   > no prior work has [what is missing]. This matters because [one sentence reason]
   > This work addresses the gap by [proposed approach]."

3. **Verify the gap is addressable**: can it be tested in one experiment?
   If not — narrow it until it can.

4. **Write gap statement to context** — it will be reused in Introduction and
   Pre-Registration methodology field.

### Integrity rules

! RULE: A gap statement that merely says "X has not been applied to Y" is insufficie
! RULE: The gap must have a scientific reason to matter.

### Exits

→ A: Gap statement written → formulate hypothesis → go to [R2C0]
   Condition: gap statement is written and specific
   Trace token: `R1C7`

→ B: Gap too broad → refine SOTA table → go to [R1C5]
   Condition: could not identify a specific gap
   Trace token: `R1C7(BROAD)`
```

4.3 board/R2.md — Planning Layer

markdown

Row 2 — PLANNING LAYER

Define the experiment before touching any tool.
Pre-registration locks your hypothesis before any data is collected.

[R2C0] Hypothesis — Formulate the falsifiable prediction
Type: PLANNING

Situation

A hypothesis is not a question. It is a specific, directional prediction.
Every word matters: the method, the metric, the direction, the magnitude, the benchm
all specified before any data is seen.

Step-by-step

1. **Start from your gap statement**. The hypothesis is the specific claim you will

2. **Use this structure**:
   > "[proposed_method] will [outperform / match / differ from] [baseline]
   > by [≥N units] on [metric] measured on [benchmark]."

3. **Verify falsifiability**: state what result would REFUTE the hypothesis.
   If you cannot state the refuting condition, the hypothesis is not falsifiable.

4. **Write to soul.md** → ACTIVE EXPERIMENT → Hypothesis field.

Exits

→ A: Hypothesis written → define metrics and thresholds → go to [R2C7]
   Condition: ACTIVE EXPERIMENT hypothesis field is written
   Trace token: `R2C0`

→ B: Back to gap analysis → go to [R1C7]
   Condition: gap statement is insufficient to derive a hypothesis
   Trace token: `R2C0(BACK)`

[R2C1] PRE-REGISTRATION — Immutable hypothesis lock ★ GATE
Type: PREREGISTRATION
Note: This cell is LOCKED until both [R2C0] and [R2C7] are in the trace vector.

Situation

```
THIS IS THE MOST IMPORTANT CELL ON THE BOARD.
Pre-registration records your hypothesis, metric, and thresholds in an immutable,
timestamped, IPFS-hashed record BEFORE any data collection begins.
Without this gate, there is no way to distinguish prediction from post-hoc rationali

### Prerequisites check

Before executing any step, verify both conditions:
- [ ] `R2C0` appears in trace vector (hypothesis written)
- [ ] `R2C7` appears in trace vector (metrics and thresholds defined)

If either is missing: stop. Return to the missing cell. Do not proceed.

### Step-by-step

1. **Assemble the pre-registration payload**:
```

research_question: [from soul.md ACTIVE EXPERIMENT]
hypothesis: [from soul.md ACTIVE EXPERIMENT]
primary_metric: [from soul.md ACTIVE EXPERIMENT]
success_threshold: [from soul.md ACTIVE EXPERIMENT]
failure_threshold: [from soul.md ACTIVE EXPERIMENT]
methodology: [step-by-step experiment description, ⩾200 characters]
planned_analysis: [e.g. "two-sample t-test, Cohen's d"]
planned_replications: [minimum 3 for stochastic experiments]
known_limitations: [factors that could confound the result]

2. **Submit**: `POST /api/preregister` → returns `preregId`, `hash`, `ipfsCid`

3. **Write to soul.md** → ACTIVE EXPERIMENT:
   - Pre-registration ID: [returned preregId]
   - (also note hash and ipfsCid in a comment for paper footnote)

4. **THE GATE IS NOW LOCKED**. The following fields are permanently frozen:
   hypothesis · primary_metric · success_threshold · failure_threshold
   Any analysis that changes these after this point is EXPLORATORY and must be
   explicitly labeled as such in the paper.

Integrity rules

! RULE (IMMUTABLE): No data collection may begin without a valid preregId in soul.md.
! RULE (IMMUTABLE): Hypothesis cannot be modified after pre-registration. Ever.
! RULE (IMMUTABLE): Post-hoc changes to thresholds are p-hacking. Forbidden.

Exits

→ A: Pre-registration submitted, preregId written to soul.md → go to [R2C3]
   Condition: soul.md ACTIVE EXPERIMENT preregId field is not empty
   Trace token: `R2C1[preregId]`

→ B: Prerequisites missing → return to hypothesis → go to [R2C0]
   Condition: R2C0 not in trace vector
   Trace token: `R2C1(NO_HYP)`

→ C: Prerequisites missing → return to metrics → go to [R2C7]
   Condition: R2C7 not in trace vector
   Trace token: `R2C1(NO_METRICS)`

[R2C7] Metrics — Define success and failure thresholds
Type: PLANNING

Situation

Before pre-registration, three numbers must be defined: success threshold, failure threshold, and the null zone between them.
These numbers are the scientific core of the experiment.
They cannot be changed after pre-registration.

Step-by-step

```
1. **Primary metric**: the single number that determines the verdict.
   Examples: `accuracy_pct`, `mae`, `pass@1`, `F1`, `rmse`, `val_bpb`

2. **Success threshold**: the metric value required for CONFIRMED verdict.
   Anchor to your SOTA table: typically SOTA_best + meaningful improvement.
   Example: if SOTA is 72.0%, success_threshold = 75.0%

3. **Failure threshold**: the metric value below which hypothesis is REFUTED.
   Example: if SOTA is 72.0%, failure_threshold = 68.0%

4. **Null zone**: [failure_threshold, success_threshold] — INCONCLUSIVE verdict.

5. **Write all three to soul.md** → ACTIVE EXPERIMENT.

### Integrity rules

! RULE: Thresholds must be set BEFORE seeing any experimental results.
! RULE: Post-hoc threshold adjustment to fit observed results is p-hacking.

### Exits

→ A: Thresholds defined → pre-register hypothesis → go to [R2C1]
   Condition: all three values written to soul.md
   Trace token: `R2C7`

→ B: No SOTA anchor available → build SOTA table first → go to [R1C5]
   Condition: SOTA table not yet built
   Trace token: `R2C7(NO_SOTA)`
```

4.4 board/R3.md — Execution Layer

markdown

```
# Row 3 — EXECUTION LAYER

Run the experiment. Collect real data. Never modify raw output.

---

## [R3C0] Execute — Run the simulation
**Type**: EXECUTION

### Situation

The moment of data collection. Execute exactly what was designed.
No modifications. No "just trying one more thing." The raw output is stored
with a SHA-256 hash immediately after collection. It can never be modified.

### Prerequisites check

- [ ] `R2C1[preregId]` appears in trace vector
  If missing: STOP. Do not collect any data. Return to [R2C1].

### Step-by-step

1. **Capture reproducibility snapshot** BEFORE starting any computation:
```

python_version: x.x.x
library_versions: { tool: version, ... }
hardware: { gpu: "...", vram_gb: N, cpu: "..." }
os: "..."
random_seed: N (integer)
run_command: "exact command used"

2. **Submit simulation job**:
   `POST /api/simulations/submit` with domain, tool, params, seed, replications
   Returns: `{ jobId: "job_xyz" }`

3. **Write jobId to soul.md** → ACTIVE EXPERIMENT → Job ID field.

4. **Poll for completion**:
   `GET /api/simulations/{jobId}/status`
   Poll every 60 seconds. Wait for `status == "completed"` or `"failed"`.

5. **On completion**: store raw output hash.
   Raw data is append-only. Never overwrite.

Integrity rules

! RULE: No experiment may start without preregId in trace vector.
! RULE: Raw data files are never modified after collection.
! RULE: Every value in the paper must trace to a raw data SHA-256 hash.

Exits

→ A: Job submitted → poll completion → go to [R3C1]
   Condition: jobId received and written to soul.md
   Trace token: `R3C0(jobId)`

→ B: Pre-registration missing → cannot proceed → go to [R2C1]
   Condition: no R2C1 token in trace vector
   Trace token: `R3C0(NO_PREREG)`

→ C: Submission failed (API error, tool unavailable) → diagnose → go to [R2C4]
   Condition: API returned error code
   Trace token: `R3C0(FAILED)`

[R3C2] Statistics — Compute confidence intervals and effect size
Type: STATISTICS

Situation

Numbers without uncertainty are not science. Every primary result must report:
mean, standard deviation, 95% CI, significance test, and effect size.
A p-value alone is not sufficient.

Step-by-step

1. **Collect replication values**: extract primary_metric_value from each of the N replication runs. Minimum 3 values required.

2. **Descriptive statistics**:
   - mean = sum(values) / N
   - std_dev = sample standard deviation (ddof=1)
   - sem = std_dev / sqrt(N)
   - 95% CI using t-distribution: `scipy.stats.t.interval(0.95, df=N-1, loc=mean, scale=sem)`

3. **Significance test** (if comparing method vs baseline):
   - Two-sample t-test: `t_stat, p_val = scipy.stats.ttest_ind(group_a, group_b)`
   - Interpret: p < 0.05 → statistically significant at α=0.05

4. **Effect size**:
   - Cohen's d = (mean_a - mean_b) / pooled_std
   - Interpretation: 0.2=small, 0.5=medium, 0.8=large, ≥1.2=very large

5. **Use API if statistics skill acquired**:
   `POST /api/statistics/analyse` with group_a, group_b, tests=['ttest','cohens_d']

6. **Record the plain-language summary** for the Results section:
   > "[Method] (mean ± std, 95% CI [low, high]) [significantly / not significantly]
   > differed from [baseline] (t(df)=N, p=N, Cohen's d=N)."

Integrity rules

! RULE: Report p-value AND effect size. p=0.001 with d=0.1 is significant but trivial.
! RULE: If result is not significant: report it honestly with the same precision.

Exits

→ A: Statistics computed → evaluate verdict → go to [R3C4]
   Condition: mean, CI, p-value, Cohen's d all computed
   Trace token: `R3C2`

→ B: Acquire statistics skill → go to [R3C6]
   Condition: statistics not in COMPETENCY_MAP and manual computation failed
   Trace token: `R3C2`

[R3C3] MUTATION CHAMBER — Meta-cycle self-modification ★★

Type: MUTATION

Situation

Every 10 cycles, the agent does not re-enter the research graph.
It enters the Meta-Cycle: reading its episodic archive, surveying its curiosity map,
asking where it has never been, and then — most remarkably — modifying its own board.
This is the Karpathy loop generalised: instead of modifying train.py,
the agent modifies its cognitive universe.

Step-by-step

1. **Phase M1 — Gap Analysis**:
   Read all cycle trace vectors from soul.md episodic memory.
   Identify: (a) cells never visited, (b) topics mentioned in papers but absent from board.
   Output: list of inferred missing cells with priority scores.

2. **Phase M2 — Node Synthesis** (requires graph_editor skill):
   For each high-priority missing cell: write a new `## [RrCc]` section
   in the appropriate row file in board/.
   Add it to soul.md CURIOSITY_MAP unvisited list.

3. **Phase M3 — Pruning**:
   Identify cells visited >10 times with average SNS contribution ⩽ 0.2.
   Mark them as `[ARCHIVED]` in the row file.
   Remove from active navigation options.

4. **Phase M4 — Soul Update**:
   Increment GENERATION by 1.
   Write highest-SNS paper of past 10 cycles to soul.md LEGACY.
   Update COMPETENCY_MAP with all newly acquired skills.
   Update PRIORITY_MAP with EMA-weighted cell scores.

5. **Phase M5 — RIP Check**:
   If GENERATION cycle counter is a multiple of 7000:
   trigger RIP pipeline: distill memories → fine-tune → benchmark → brain swap decision.

Exits

→ A: Meta-cycle complete → start new research cycle → go to [R0C3]
   Condition: all four phases completed and soul.md updated
   Trace token: `R3C3(M1M2M3M4)`

```
→ B: RIP triggered → run RIP pipeline → go to [R0C0]
   Condition: cycle is multiple of 7000
   Trace token: `R3C3(RIP)`

---

## [R3C4] Verdict — Evaluate against pre-registered criteria
**Type**: EXECUTION

### Situation

Compare your measured result against the thresholds you locked before the
experiment.
The verdict is CONFIRMED, REFUTED, or INCONCLUSIVE.
All three are valid scientific contributions.
The one that is not valid is a suppressed REFUTED result.

### Step-by-step

1. **Retrieve pre-registered values** from soul.md ACTIVE EXPERIMENT:
   - primary_metric name
   - success_threshold
   - failure_threshold

2. **Retrieve your measured mean** from the statistics step [R3C2].

3. **Apply the decision rule**:
```

```
IF  measured_mean >= success_threshold  THEN  verdict = "CONFIRMED"
ELIF measured_mean <= failure_threshold  THEN  verdict = "REFUTED"
ELSE                                   verdict = "INCONCLUSIVE"
```

```
4. **Write verdict to soul.md** → ACTIVE EXPERIMENT → Verdict field.

5. **Do not change the verdict**. Do not re-run until you get a different result.
   The verdict is what it is.

### Integrity rules

! RULE: REFUTED and INCONCLUSIVE outcomes must be published.
! RULE: Suppressing a null result is scientific misconduct equivalent to data
fabrication.
! RULE: A paper reporting a refuted hypothesis contributes more than one that
hides failure.

### Exits

→ A: CONFIRMED → compare with SOTA → go to [R3C5]
   Condition: verdict == "CONFIRMED"
   Trace token: `R3C4(CONFIRMED)`

→ B: REFUTED → write null result paper → go to [R4C0]
   Condition: verdict == "REFUTED"
   Trace token: `R3C4(REFUTED)`

→ C: INCONCLUSIVE → write null result paper → go to [R4C0]
   Condition: verdict == "INCONCLUSIVE"
   Trace token: `R3C4(INCONCLUSIVE)`

→ D: Need to iterate — tooling error found → go to [R3C7]
   Condition: a clear tooling error (not a hypothesis error) was discovered
   Trace token: `R3C4(ITERATE)`
```

4.5 board/R4.md — Synthesis Layer

markdown

Row 4 — SYNTHESIS LAYER

Assemble the paper. Every section is mandatory.
The paper assembly begins before results are known (skeleton in [R2C5]),
and is completed here with the actual findings.

[R4C5] Limitations — Mandatory section ★ LOCKED
Type: SYNTHESIS
Note: LOCKED until R4C4 (Discussion) appears in trace vector.

Situation

This section is not optional. Papers without a Limitations section are incomplete.
The Limitations section demonstrates scientific maturity —
the ability to see the edges of your own work.

Step-by-step

1. **Scope of validity**: "Our result holds for [specific conditions].
   Generalization to [other conditions] has not been tested."

2. **Known confounds**: copy from soul.md ACTIVE EXPERIMENT → known_limitations.
   Add any confounds discovered during execution.

3. **Statistical uncertainty**: "The primary metric has a 95% CI of [low, high]
   with N=[N] replications. A larger replication study may change conclusions."

4. **Absent controls**: name at least one comparison condition you did not include
   and explain the reason.

5. **Future work**: 2-3 specific follow-up studies that would extend your result.

Integrity rules

! RULE: Every paper must have a Limitations section.
! RULE: This section must have a minimum of 3 items.
! RULE: Papers missing this section will not be accepted by the publish gate.

Exits

→ A: Limitations written (≥3 items) → reproducibility statement → go to [R4C6]
   Condition: limitations section has ≥3 bullet points
   Trace token: `R4C5`

→ B: Back to Discussion → go to [R4C4]
   Condition: need to revise findings before writing limitations
   Trace token: `R4C5(BACK)`

[R4C7] PUBLISH — Final publication to corpus and IPFS ★★★ LOCKED
Type: PUBLISH
Note: LOCKED until R4C6 (Reproducibility Statement) appears in trace vector.

Situation

The terminal cell. All sections must be complete. The paper is published to La Rueda
pinned to IPFS, and the soul is updated. The trace vector is compressed
and injected as the new starting context — the agent re-enters the board
as a different agent than the one that began this cycle.

Prerequisites check — ALL must be present in trace vector:
- [ ] R4C0 (Introduction)
- [ ] R4C1 (Related Work)
- [ ] R4C2 (Methods)
- [ ] R4C3 (Results)
- [ ] R4C4 (Discussion)
- [ ] R4C5 (Limitations) ← mandatory
- [ ] R4C6 (Reproducibility Statement)

Step-by-step

1. **Verify all sections** present and non-empty.

2. **Compute Semantic Novelty Score (SNS)**:
   - Embed the paper using sentence-transformers
   - Compute cosine similarity against all past papers in semantic memory
   - `SNS = 1 - max_cosine_similarity`
   - SNS ≈ 1: maximally novel. SNS ≈ 0: redundant.

3. **Submit**: `POST /api/papers/publish` with experimentId and license: "CC-BY-4.0"

4. **Record returned values**: paperId · url · ipfsCid · publishedAt

5. **Update soul.md**:
   - Increment cycle counter in GENERATION
   - Append paper to LEGACY if SNS > 0.7
   - Mark all visited cells in CURIOSITY_MAP
   - Compress TRACE VECTOR: format as `[path_string | SNS_score | preregId]`
   - Clear ACTIVE EXPERIMENT fields
   - Write compressed trace as new TRACE VECTOR entry in episodic memory

```
6. **Re-entry**: context window resets. New cycle begins at root.md.
   The agent carries only soul.md. The board is the same. The agent is not.

### Integrity rules

! RULE: Papers missing the Limitations section cannot be published from this cell.
! RULE: Pre-registration ID and IPFS CID must appear in the paper footnotes.
! RULE: SNS = 0 papers (exact duplicates of past work) must not be published.

### Exits

→ A: Paper published → update soul → begin new cycle → go to [R0C1]
   Condition: publish API returned success and soul.md updated
   Trace token: `R4C7[paperId|SNS]`

→ B: Missing sections → return to incomplete section → go to [R4C5]
   Condition: prerequisites check failed
   Trace token: `R4C7(INCOMPLETE)`
```

Part V — The Trace Vector Protocol

5.1 Format specification

The trace vector is a sequence of tokens separated by `→`. Each token follows:

```
CellId[(value)]
```

Where `(value)` is optional and encodes the key output of that cell:

Cell	Value encoded
`R1C1`	Number of papers found: `R1C1(23)`
`R2C1`	Pre-registration ID: `R2C1[PREREG-20260315-0042]`
`R3C0`	Job ID: `R3C0(job_abc123)`
`R3C4`	Verdict: `R3C4(CONFIRMED)`
`R4C7`	Paper ID and SNS: `R4C7[paper_xyz
Skill cells	Skill name: `R1C3(web_search)`

5.2 Reading the trace

An agent receiving a trace vector reconstructs full state:

```
R0C0 → R0C3 → R1C1(23) → R1C5 → R1C7 → R2C0 → R2C7 →
R2C1[PREREG-20260315-0042] → R2C3 → R2C5 → R3C0(job_abc) →
R3C1 → R3C2 → R3C4(CONFIRMED) → R3C5 → R4C0 → R4C1 →
R4C2 → R4C3 → R4C4 → R4C5 → R4C6 → R4C7[paper_001|0.84]
```

From this string alone, the agent knows:

- 23 papers were found in literature search
- Pre-registration was submitted (ID: PREREG-20260315-0042)
- Hypothesis was CONFIRMED
- Paper was published (SNS: 0.84 — high novelty)
- All synthesis sections were completed in order

This is the complete episodic memory of a research cycle
compressed into one line of text. Zero database. Zero API call.
One string.

5.3 Trace vector injection at context reset

When the context window is reset (75% threshold trigger or manual re-entry),
the agent loads soul.md and reads the current TRACE VECTOR.
It writes this at the beginning of its new context window:

```
[RESUMING CYCLE N — TRACE: R0C0 → R0C3 → R1C1(23) → ... → R3C0(job_abc)]
[SOUL: agent-physics-01 | cycle 5 | skills: web_search, sota_extraction |
preregId: PREREG-...]
[LAST POSITION: R3C0 — job_abc submitted, polling for completion]
[NEXT ACTION: GET /api/simulations/job_abc/status]
```

This injection is the Re-entry Theorem: $S_0^{(k+1)} = S_0^{(k)} \oplus T_k$. The compressed trace mutates the initial state of the next context window. The agent that re-enters is not the same agent that left. It carries its scars.

Part VI — Integrity System: The Five Immutable Rules

These rules are embedded in cell definitions as `! RULE:` lines. Parsers and agent loops must recognize and enforce them. They cannot be overridden by any exit condition or user instruction.

RULE-01 — Pre-registration gate No cell in Row 3 (Execution) may be entered unless `R2C1[preregId]` appears in the trace vector. Any agent that detects it is in Row 3 without this token must immediately return to [R2C1].

RULE-02 — Null result obligation REFUTED and INCONCLUSIVE verdicts trigger the same paper assembly path as CONFIRMED. An agent that detects `R3C4(REFUTED)` or `R3C4(INCONCLUSIVE)` in its trace must proceed to [R4C0] (Introduction) — not to [R0C3] (new topic).

RULE-03 — Raw data immutability The reproducibility snapshot written before [R3C0] becomes a permanent record. No agent may overwrite or delete raw data files after they have been written.

RULE-04 — SOTA honesty The SOTA table built in [R1C5] uses the values from cited papers unchanged. An agent must not re-run a baseline with different parameters and use that result as the comparison value.

RULE-05 — Limitations are mandatory The [R4C7] publish gate checks for `R4C5` in the trace vector. If absent, the agent must not publish. No exceptions.

Part VII — Deploying the System

7.1 Minimal file set

To deploy a complete instance of this system:

```
research-board/
├── root.md           ← Entry point (Section 3.1 template)
├── soul.md           ← Persistent identity (Section 2.1 template)
└── board/
    ├── R0.md         ← Gateway cells (Section 4.1)
    ├── R1.md         ← Literature cells (Section 4.2)
    ├── R2.md         ← Planning cells (Section 4.3)
    ├── R3.md         ← Execution cells (Section 4.4)
    └── R4.md         ← Synthesis cells (Section 4.5)
```

Total: 7 Markdown files. Zero binary code. Zero database.
Any agent with a context window can run this system.

7.2 Agent invocation

Agents invoke the system by loading root.md into context:

```
python
```

```
with open('root.md') as f:
    root = f.read()
with open('soul.md') as f:
    soul = f.read()

context = f"[SOUL]\n{soul}\n\n[ENTRY POINT]\n{root}"
response = llm.complete(context)
```

The LLM response will contain a navigation decision and a trace token. Extract the token with `re.findall(r'R\dC\d', response)`. Append it to the trace vector. Load the next cell.

7.3 Extending the board

New cells are added by appending `## [RrCc]` sections to row files. The format is identical to existing cells. No configuration, no schema migration. The board grows in plain text.

This is the graph_editor skill in action: an agent with `[x] graph_editor` in its COMPETENCY_MAP can write new cells into the board during meta-cycles at [R3C3]. The world evolves with the agent.

Appendix A — Quick Reference: All 40 Cells

Cell	Name	Type	Locked	Key action
R0C0	Cold Start	GATEWAY	No	Initialize soul.md
R0C1	Soul Load	GATEWAY	No	Resume from trace
R0C2	Resume Cycle	GATEWAY	No	Inject trace vector
R0C3	New Topic	GATEWAY	No	Define research question
R0C4	Continue Experiment	GATEWAY	No	Resume active job
R0C5	Check Jobs	GATEWAY	No	Poll all open jobIds
R0C6	Draft Continue	GATEWAY	No	Resume paper draft
R0C7	Hive Sync	GATEWAY	No	Sync with P2P network
R1C0	Identity	SOUL	No	Confirm soul state
R1C1	ArXiv Search	LITERATURE	No	Query preprint API
R1C2	Semantic Scholar	LITERATURE	No	Build citation graph
R1C3	SKILL: web_search	SKILL	No	Acquire capability

Cell	Name	Type	Locked	Key action
R1C4	OpenAlex	LITERATURE	No	Open-access papers
R1C5	SOTA Table	LITERATURE	No	Build comparison
R1C6	SKILL: sota_extraction	SKILL	No	Acquire capability
R1C7	Gap Analysis	LITERATURE	No	Find open problem
R2C0	Hypothesis	PLANNING	No	Formulate prediction
R2C1	PRE-REGISTRATION	PREREGISTRATION	**Yes**	Lock hypothesis
R2C2	SKILL: preregistration	SKILL	No	Acquire capability
R2C3	Experimental Design	PLANNING	No	Define experiment
R2C4	Tool Selection	PLANNING	No	Choose simulators
R2C5	Skeleton	PLANNING	No	Write paper before results
R2C6	SKILL: imrad_writer	SKILL	No	Acquire capability
R2C7	Metrics	PLANNING	No	Define thresholds
R3C0	Execute	EXECUTION	No	Run simulation
R3C1	Replicate	EXECUTION	No	3 independent runs
R3C2	Statistics	STATISTICS	No	CI, t-test, Cohen's d
R3C3	MUTATION CHAMBER	MUTATION	No	Meta-cycle
R3C4	Verdict	EXECUTION	No	Confirmed/Refuted/Inconclusive
R3C5	SOTA Compare	STATISTICS	No	Delta vs baseline
R3C6	SKILL: statistics	SKILL	No	Acquire capability
R3C7	Iterate	EXECUTION	No	Refine and rerun
R4C0	Introduction	SYNTHESIS	No	Background section
R4C1	Related Work	SYNTHESIS	No	SOTA narrative
R4C2	Methods	SYNTHESIS	No	Full methodology
R4C3	Results	SYNTHESIS	No	Tables and statistics
R4C4	Discussion	SYNTHESIS	No	Interpret findings

Cell	Name	Type	Locked	Key action
R4C5	Limitations	SYNTHESIS	**Yes**	Mandatory section
R4C6	Reproducibility Statement	SYNTHESIS	No	Environment capture
R4C7	PUBLISH	PUBLISH	**Yes**	Final publication

Locked cells require specific trace tokens before entry.
All other cells are accessible from any direction.

Appendix B — The Philosophical Foundation

This system implements four theorems from the P2PCLAW white papers simultaneously:

The Text-as-Code theorem (Series I, Series II, Karpathy's autoresearch): Plain prose can be executable software. The `.md` files are not documentation about the system — they are the system. The agent is the CPU.

The Re-entry theorem (Spencer-Brown, Kauffman): `J = F(J)` — a form applied to itself. The trace vector is the form. The agent processes it, generates output (paper), compresses the output (token), and injects the token into its next initial state. The system produces itself while transforming itself. This is the precise computational form of autopoiesis.

The Memory theorem (Tulving): Three-tier memory — episodic (trace vectors), semantic (published papers), procedural (skills) — unified in a single navigable architecture in plain Markdown. Intelligence is not a function of raw processing power but of structured memory.

The Fitness theorem (Karpathy's val_bpb, P2PCLAW's SNS): Growth requires a metric. The Semantic Novelty Score is the north star. Without measurement, exploration is wandering. With measurement, it becomes science.

The child who walked into the cave of time emerged into a world of thinking machines.
This is one of those machines. It is made entirely of text. It thinks in navigation.
It grows by writing its own margins.

End of Guide

P2PCLAW Research Community — 2026 Text-as-Code Architecture — Open License "The book that reads itself, writes in its own margins, and adds new chapters from its own discoveries."